

JAVA INTERVIEW PREPARATION

CHAPTER 53

HIGH-THROUGHPUT JAVA APIs

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

High-Throughput Java APIs: Capacity, Concurrency, Backpressure, and Predictable Latency

Senior / Lead Java Developer Reading Chapter

A high-throughput API is not simply an endpoint that survives a benchmark. It must meet a defined latency and availability objective under realistic traffic, dependency behavior, payload distribution, and failure. Senior system design connects workload math, admission control, Java concurrency, connection capacity, data access, and observability into one bounded system.

Begin with Requirements and Invariants

Clarify the operation before choosing technology:

```
traffic:         average, peak, burst, regional distribution
latency SLO:     p50, p95, p99, timeout budget
availability:    eligible successes over a time window
payload:         typical and maximum request/response size
consistency:     read-your-write, stale tolerance, transaction scope
correctness:     idempotency, ordering, uniqueness, authorization
growth:          present and forecast data and traffic
```

Throughput without correctness is not success. A payment endpoint processing twice as many requests by dropping idempotency is defective. State the invariant and degraded behavior before optimizing.

Estimate Capacity

Translate product numbers into resource demand. If peak traffic is 5,000 requests per second and each request performs three database statements, the database sees roughly 15,000 statements per second before retries, background jobs, or N+1 queries.

Little's Law relates average concurrency, throughput, and time in system:

```
concurrency = arrival rate x average time in system

5,000 requests/s x 0.2 s = about 1,000 in-flight requests
```

Tail latency and bursts require headroom. This estimate reveals whether thread, connection, memory, and downstream limits are plausible. Recalculate for degraded latency: if response time rises from 200 ms to two seconds, in-flight demand grows tenfold at the same arrival rate.

Draw the Resource Path

```
client
-> load balancer queue
-> server connection
-> request execution
-> database pool or HTTP client pool
-> downstream queue and service
-> serialization
-> network response
```

Every queue consumes time and memory. Measure connection acquisition separately from execution. An endpoint reported as "database 20 ms" can still spend 800 ms waiting for a pooled connection.

Avoid unbounded queues. They convert overload into growing latency and heap consumption. A bounded queue or concurrency limit rejects early, allowing callers to retry according to policy while the service preserves useful work.

Latency Budgets and Deadlines

Start from the caller-visible deadline and allocate time to each stage:

total budget:	500 ms
admission and local:	40 ms
database:	120 ms
two downstream calls:	220 ms total
serialization/network:	40 ms
recovery margin:	80 ms

The numbers are service-specific, but the budget prevents each dependency from independently consuming the full deadline. Propagate remaining time across internal calls and stop work when its answer is no longer useful.

Connect, pool-acquisition, response-header, body-read, lock, statement, and total-operation timeouts protect different phases. Configure and observe them separately.

Platform Threads, Reactive I/O, and Virtual Threads

Platform-thread pools bound concurrent work but blocked I/O occupies scarce OS threads. Reactive frameworks represent waiting through callbacks and can support high concurrency with fewer threads, but require non-blocking dependencies and disciplined pipelines.

Virtual threads make thread-per-request or thread-per-task practical for blocking I/O. They simplify imperative code and reduce blocked-thread cost. They do not make CPU work faster or create database connections, memory, remote quotas, or network bandwidth.

```
10,000 virtual requests
-> semaphore allows 100 pricing calls
-> database pool allows 60 transactions
-> excess requests wait only within bounded deadline or are rejected
```

Choose the programming model that fits libraries and team expertise. Capacity limits remain explicit in every model. Avoid mixing blocking calls into an event loop and avoid wrapping already non-blocking work in unnecessary executors.

CPU-Bound Versus I/O-Bound Work

CPU-bound work such as compression, encryption, image processing, or complex rules is limited by cores. Increasing threads beyond useful parallelism adds scheduling and cache contention.

I/O-bound work spends time waiting and can sustain more concurrency, but downstream capacity becomes the limit. Profile before categorizing. JSON serialization, allocation, and logging can become CPU-bound at high request rates even when business work is mostly I/O.

Separate large CPU tasks from latency-sensitive request executors. Use bounded executors or asynchronous jobs so one expensive request cannot occupy every core.

Admission Control and Load Shedding

Admission control rejects work before saturated resources accept it. Limits can apply globally, per endpoint, tenant, client, priority, or dependency.

```
incoming request
-> authenticate identity
-> enforce tenant/global rate
-> enforce concurrency and queue limit
-> perform work
```

Rate limiting controls work over time; concurrency limiting controls work in flight. Both may be necessary. Return 429 for client quota policy and 503 for temporary service capacity according to the contract, with **Retry-After** only when meaningful.

Preserve priority. Health checks, control operations, and critical writes may need reserved capacity so bulk reporting traffic cannot starve recovery.

Thread Pools and Queue Design

An executor is a resource owner. Name it for workload, bound its threads and queue, define rejection, and expose metrics. One shared executor allows a slow optional dependency to starve unrelated operations.

```
interactive request executor
pricing-call bulkhead
report-generation executor
event-publication executor
```

Large queues hide saturation until latency is already unacceptable. Small bounded queues plus fast rejection make overload visible. Caller-runs rejection can propagate backpressure but is dangerous on event loops or thread-constrained request paths.

Virtual-thread executors need no pool to ration threads, but semaphores and connection pools must ration scarce downstream access. ThreadLocal values multiplied across huge virtual-thread counts can consume substantial memory.

Database Capacity

Total database connections equal per-instance pool size multiplied by application replicas plus jobs and operational headroom. Autoscaling application pods without coordinating pools can overwhelm the database.

Use efficient indexed queries, explicit fetch plans, keyset pagination, bounded result sets, and short transactions. Batch related statements when semantics allow. Avoid holding connections across remote calls.

Read replicas can offload tolerant reads but introduce lag and do not scale writes. Sharding partitions write and data ownership at much higher operational cost. Exhaust indexing, query, schema, cache, and vertical capacity evidence before introducing sharding.

Downstream Call Graphs

Sequential calls add latency. Parallel fan-out shortens the critical path only when calls are independent, while increasing simultaneous load.

```
API request
-> customer service
-> pricing service
-> inventory service
```

Three dependencies reduce combined availability and increase tail risk. Avoid deep chatty graphs. Replicate read models or change boundaries when one response repeatedly requires many remote calls.

Retries multiply traffic. Centralize retry ownership, require idempotency, apply jittered backoff and retry budgets, and stop when the request deadline is nearly exhausted. Circuit breakers fail fast during sustained

dependency failure; bulkheads protect caller capacity.

Caching

Cache data only when reuse and acceptable staleness justify consistency cost. Local caches offer low latency but differ across instances. Distributed caches share values but add a network dependency.

Define key, value contract, TTL, eviction, invalidation, tenant isolation, maximum value size, stampede behavior, and cache-failure fallback. Cache hit ratio alone is not success; measure database work avoided and correctness.

Do not cache unbounded user-specific combinations. Personalized keys can create enormous cardinality and low reuse.

Serialization and Payload Control

Large payloads consume network, heap, CPU, and garbage-collection capacity. Limit request size, nesting, collection cardinality, and response page size before expensive work.

Select only required database columns and map to DTOs. Streaming avoids buffering the entire response but holds connections and resources longer and complicates error handling after commitment.

Compression reduces bandwidth but costs CPU and can worsen latency for small responses. Configure thresholds and benchmark representative content. Avoid logging complete payloads.

Memory and Allocation

At high request rate, small per-request allocations multiply. A 100 KB temporary graph at 10,000 requests per second creates about 1 GB/s allocation before framework overhead.

Profile allocation rate and retaining paths. Avoid accidental materialization of huge JPA graphs, unbounded collectors, duplicated byte arrays, and large ThreadLocal context. Object pooling is rarely the first answer; it introduces contention and lifecycle risk. Reduce work and bound payloads first.

GC pauses depend on live set, allocation, collector, heap and container limits. Monitor request latency alongside allocation and GC, not only pause duration averages.

Idempotency and High-Throughput Writes

Retries, client timeouts, and duplicate submissions are normal. For retryable creation, persist a client-scoped idempotency key, request fingerprint, state, and outcome atomically with the business effect.

The idempotency store is on the hot write path. Index it for key lookup, partition or retain according to volume, and avoid storing unbounded response bodies. Concurrent duplicates need one atomic claimant and defined in-progress behavior.

Database uniqueness and conditional updates protect invariants across instances. In-memory locks do not.

Horizontal Scaling and Statelessness

Stateless request handling allows any instance to serve a request, but the system still has state in databases, caches, sessions, brokers, and external providers. Move state only when the tradeoff improves ownership and availability.

Session affinity can improve local-cache reuse but creates uneven load and failover surprises. Server sessions may be appropriate when centrally stored or replicated according to requirements. A JWT removes session lookup but creates claim staleness and revocation tradeoffs.

Scale from the limiting resource. If database connections are saturated, adding API replicas can worsen failure. If CPU is saturated and the database has headroom, additional instances can help.

Hot Keys and Uneven Traffic

Averages hide skew. One tenant, product, or partition can dominate load. Rate-limit fairly, shard by a key with acceptable distribution, and isolate extremely large tenants when justified.

Hot cache keys cause stampedes; hot database rows cause lock contention; hot message keys limit partition parallelism. Measure top contributors safely without turning tenant IDs into unbounded metric labels.

Observability and SLOs

Use service-level indicators for eligible success and latency. Measure p50, p95 and p99 rather than average alone. Break latency into queue, execution, database, downstream, serialization and network.

Core saturation signals include executor active and queued tasks, virtual-thread counts where useful, connection acquisition, database locks, HTTP client pools, cache latency, event-loop delay, CPU, allocation, GC, and network.

Use route templates and bounded outcomes as metric tags. Traces explain representative critical paths; structured logs capture detailed failures; metrics detect fleet-wide impact.

Load Testing

Use realistic arrival patterns, payloads, data distributions, connection reuse, think time, and authentication. Closed-loop tests can understate overload because clients slow down as the service slows; open or controlled-arrival tests reveal queue growth.

Test steady state, burst, dependency slowdown, partial failure, cache cold start, deployment, autoscaling delay, and recovery. Run long enough to expose pool leakage, GC cycles, cache expiry and thermal effects.

Define pass criteria before testing: SLO, maximum error, saturation headroom, and no violated invariant. Throughput at unacceptable p99 is not capacity.

Test recovery, not only the peak

A useful performance test is a sequence rather than a single plateau. Warm the service, drive normal traffic, introduce a burst, slow one dependency, restore it, and observe whether queues drain. A system can survive the fault yet remain unhealthy because retries, pooled connections, or accumulated work continue to dominate after the dependency recovers.

The workload model should preserve production skew. Uniform random identifiers hide hot tenants and hot rows. Reusing one authenticated session can hide authentication cost, while creating a new connection for every request can exaggerate it. Record the assumptions alongside the result: payload distribution, cache state, database size, connection reuse, think time, and arrival pattern.

Graceful Degradation and Brownout Design

High availability does not always mean returning the complete response. If recommendations, counts, or decorative metadata are optional, the API can omit them when their dependency is slow. This is a deliberate brownout mode, not accidental partial data. The response contract should make absence understandable and telemetry should show when degradation is active.

<pre>normal response = required account data + optional recommendations + live counts degraded mode = required account data + cached/absent optional fields</pre>

Required and optional work need separate time budgets and preferably separate bulkheads. Otherwise an optional dependency can consume the same threads or connections needed for the core operation. Degradation decisions should be automated from measured health but bounded to avoid rapid oscillation. Recovery can be gradual so a recently restored dependency is not hit immediately by the entire backlog.

Backpressure Across Service Boundaries

Backpressure means the producer cannot indefinitely create work faster than the consumer completes it. Reactive streams express demand in-process, but an HTTP caller does not automatically understand a downstream Java publisher's demand. Across network boundaries, backpressure becomes bounded concurrency, request limits, explicit rejection, queue capacity, or a broker with controlled consumption.

An HTTP 429 `Too Many Requests` indicates caller-specific rate limiting; 503 `Service Unavailable` commonly represents temporary service-wide inability. A `Retry-After` hint is useful only when clients are designed to respect it with jitter. Returning an error quickly can preserve the service for admitted traffic and produce better overall outcomes than allowing every request to time out.

Deployment and Capacity Safety

Rolling deployment temporarily changes capacity. If ten instances normally handle the load and two are unavailable while being replaced, the remaining eight must have headroom. New instances may compile hot methods, establish pools, and warm caches before reaching steady-state throughput. Readiness should therefore reflect the ability to serve, not merely that the process has started.

Autoscaling is reactive and delayed. Scaling on CPU alone misses connection saturation and blocking latency; scaling on raw request count misses different request costs. Useful signals include queue delay, in-flight work, latency, CPU, and dependency saturation. Minimum capacity should absorb expected bursts while new instances start. A design that requires instant autoscaling is already under-provisioned.

Common Senior-Level Traps

Traps include optimizing average latency, using unbounded queues, increasing every pool, adopting reactive code with blocking dependencies, assuming virtual threads remove capacity limits, and scaling application replicas beyond database capacity.

Other traps include retry multiplication, caching without invalidation, load tests with tiny uniform data, and reporting requests per second without error and latency distributions.

A Strong Interview Answer

I begin with traffic, payload, consistency, and p99 SLO, then estimate concurrency using arrival rate and time in system. I draw every queue and pool and allocate one end-to-end deadline. The Java execution model can use platform threads, reactive I/O, or virtual threads, but downstream capacity is always bounded through pools, semaphores, admission control, and load shedding. I minimize call depth, use idempotency and conditional database writes, and cache only with an explicit stale and invalidation contract. I scale from the measured bottleneck and load-test realistic skew, bursts, dependency slowdown, cold caches, and recovery while observing queue time, connection wait, allocation, GC, and domain correctness.

Interview-Ready Summary

High throughput is predictable useful work under a latency and correctness objective. Capacity planning converts traffic and service time into concurrency and resource demand. Bounded queues, deadlines, bulkheads, efficient data access, and overload rejection keep failure contained.

Java concurrency choices change the cost of waiting but not CPU or downstream capacity. Measure the full request path and scale only the actual limiting resource.

Revision Notes

- Define traffic, payload, invariants, availability, and p99 latency first.
- Little's Law relates throughput, time in system, and concurrency.
- Degraded latency multiplies in-flight resource demand.
- Draw every queue, executor, and connection pool.
- Allocate downstream work from one end-to-end deadline.
- Timeout phases protect different waits and resources.
- Reactive I/O requires non-blocking dependencies.
- Virtual threads make blocking concurrency cheaper, not unlimited.
- Bound scarce resources with pools, semaphores, and admission control.
- Rate and concurrency limits solve different overload problems.
- Unbounded queues convert overload into latency and memory growth.
- Total database connections include every replica and job.
- Avoid remote calls while holding database transactions.
- Fan-out lowers critical-path time but increases downstream load.
- Retries need idempotency, jitter, budgets, and remaining time.
- Cache only with explicit key, freshness, invalidation, and failure policy.
- Limit payload size and allocation before expensive work.
- Database constraints protect invariants across application instances.
- Scale from the measured bottleneck, not CPU alone.
- Load-test skew, bursts, failures, cold start, and recovery.